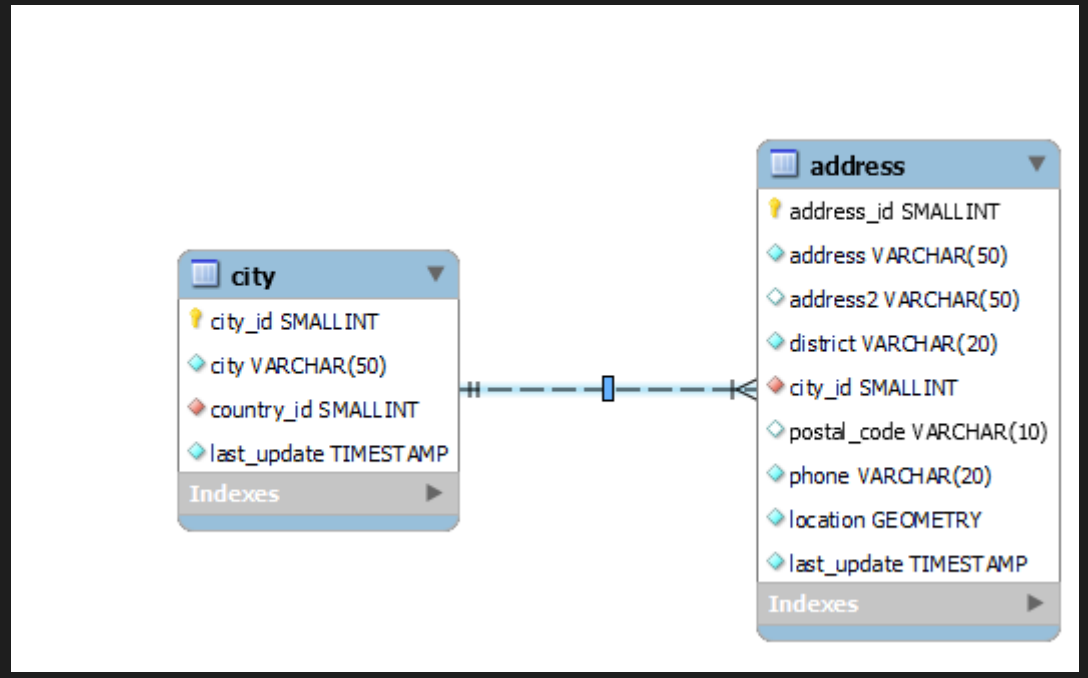
SQL – JOINS

SHENKAR

BOAZ SCHNEIDER

- בסיס נתונים מכיל מספר רב של טבלאות המכילות מידע מקושר.
- החיבור בין הטבלאות מבוצע באמצעות עמודות דומות. עמודות אלו נקראות foreign keys
- כאשר אפליקציה תרצה לתשאל את בסיס הנתונים היא תפנה בד"כ ליותר מטבלה אחת

- דוגמה לחיבור:
  חיבור בין טבלת **כתובת** (**address**) לטבלת **עיר** (**city**) – החיבור יבוצע באמצעות העמודה –city_id



- על מנת לקבל את נתוני הכתובות נצטרך לתשאל את שתי הטבלאות יחד :
  city address

- התשאול יבוצע באמצעות JOIN
- JOINהיא השיטה לחיבור בין טבלה אחת או יותר באמצעות עמודה/עמודות דומות.

- ישנם 4 סוגי חיבורים:
  - Inner join
  - Left join
  - Right join
  - Cross join

דוגמה:

1. create database:

```
CREATE DATABASE sand_box;
```

2. create two tables called *members* and *students* :

```
CREATE TABLE members (
    member_id INT
    AUTO_INCREMENT,
    name VARCHAR(100),
    PRIMARY KEY (member_id)
);
```

```
CREATE TABLE students (
    student_id INT AUTO_INCREMENT,
    name VARCHAR(100),
    PRIMARY KEY (student_id)
);
```

דוגמה (המשך):

3. insert some rows into **members** and **students** :

```
INSERT INTO members(name)
VALUES('John'),('Jane'),('Mary'),('David'),('Amelia')
;

INSERT INTO students(name)
VALUES('John'),('Mary'),('Amelia'),('Joe');
```

דוגמה (המשך):

Now, query the data from the tables **members** and **students**:

SELECT * FROM members;

member_id	name
1	John
2	Jane
3	Mary
4	David
5	Amelia

SELECT * FROM students;

student_id	name
1	John
2	Mary
3	Amelia
4	Joe

INNER JOIN

- The inner join clause joins two tables based on a condition which is known as a join predicate.

- חיבור שתי טבלאות בהתבסס על תנאי מסויים

BASIC SYNTAX OF THE INNER JOIN

- The following shows the basic syntax of the inner join clause that joins two tables table_1 and table_2:

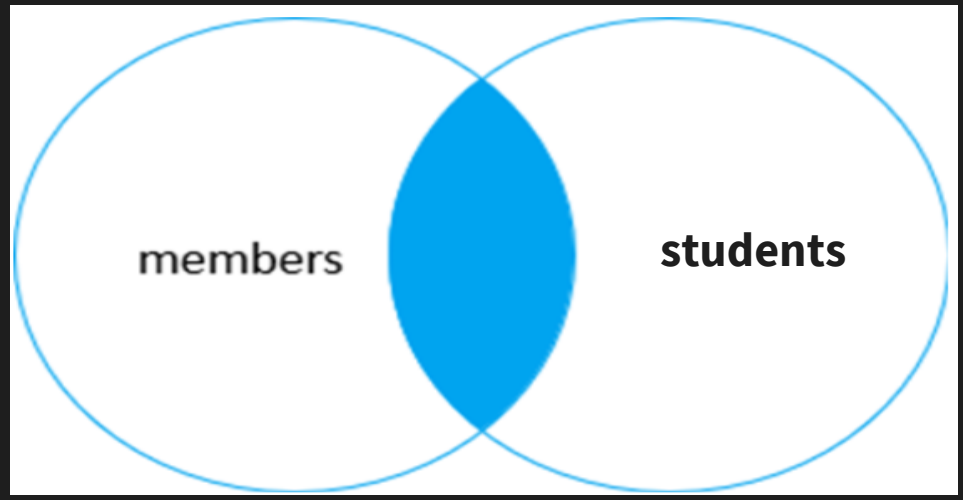
```
SELECT
column_list FROM
table_1
inner join table_2 ON join_condition;
```

- ה inner join משווה כל רשומה בין table_1 לבין table_2 על בסיס תנאי ה join
- אם הרשומות עונות על תנאי ה join (TRUE) – נוצרת רשומה שמכילה את העמודות מ 2 הטבלאות ואת הנתונים מ 2 הטבלאות
- אם הרשומות לא עונות על תנאי ה join (FALSE) - הן לא מוכללות בתוצאה הסופית

- **In other words, the inner join clause includes only rows whose values (for the join condition) match.**

The following statement finds members who are also students:

```
SELECT
        m.member_id,
        m.name memmber_name,
        c.student_id,
        c.name student_name
FROM    members AS m
INNER JOIN    students AS c
ON m.name = c.name;
```



	member_id	memmber_name	student_id	student_name
▸	1	John	1	John
	3	Mary	2	Mary
	5	Amelia	3	Amelia

THE INNER JOIN AND THE USING CLAUSE

- במידה והעמודה עליה מבוצע ה join זהה בשתי הטבלאות ניתן לבצע את ה join באמצעות המילה השמורה using:

```
SELECT
column_list  FROM
table_1
inner join table_2 using (column_name);
```

דוגמה ל JOIN עם USING:

```
SELECT
        m.member_id,
        m.name
memmber_name,
        c.student_id,
        c.name student_name
FROM    members AS m
INNER JOIN
students AS c
ON m.name = c.name;
```

```
SELECT
        m.member_id,
        m.name memmber_name,
        c.student_id,
        c.name student_name
FROM    members AS m
INNER JOIN
students AS c
USING (name);
```

דוגמה #1

```
SELECT
t1.city_id,t1.city,
t2.address_id,t2.address,t2.city_id
FROM
city t1
INNER JOIN address t2
ON t1.city_id = t2.city_id;
```

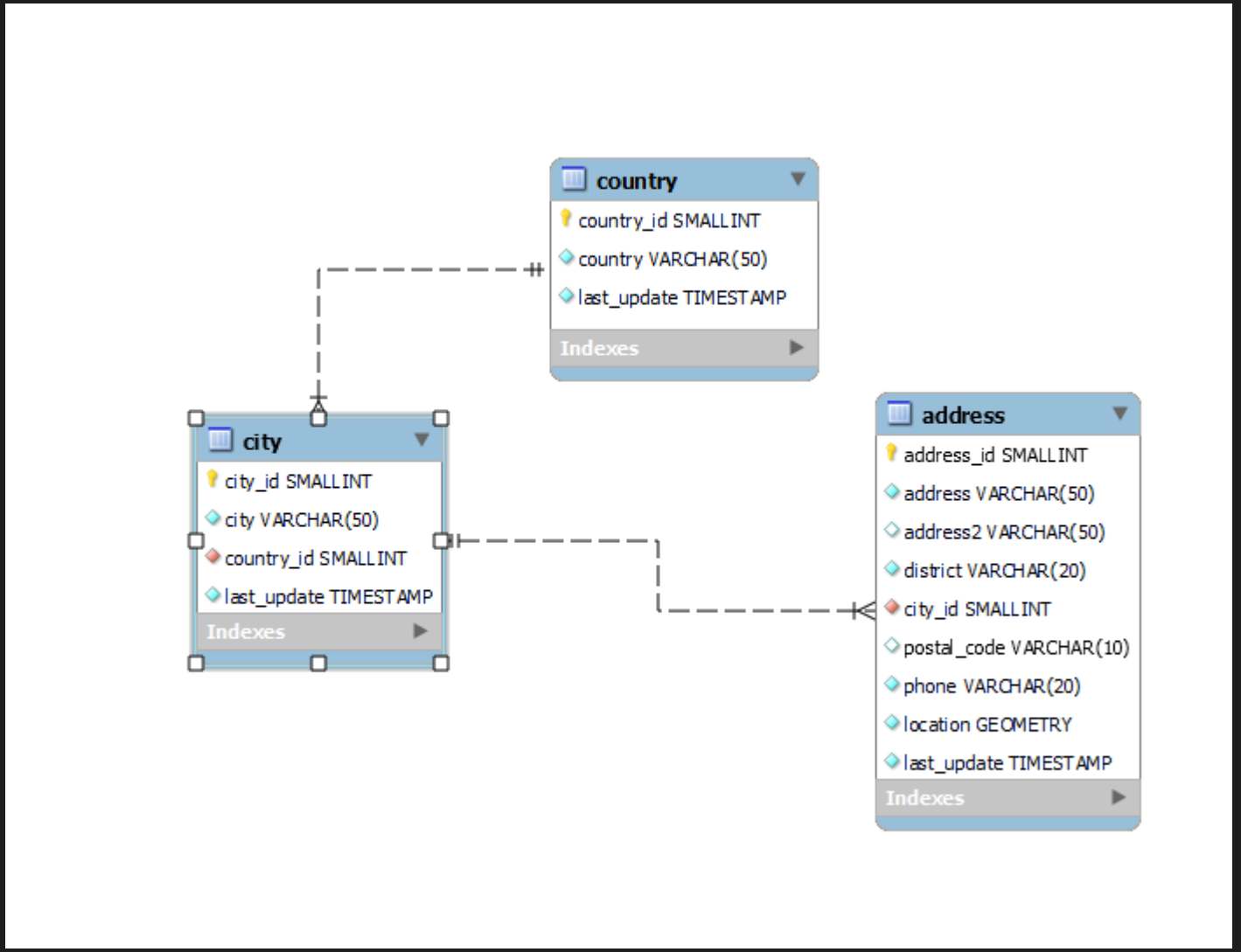
city_id	city	address_id	address	city_id
1	A Corua (La Corua)	56	939 Probolinggo Loop	1
2	Abha	105	733 Mandaluyong Place	2
3	Abu Dhabi	457	535 Ahmadnagar Manor	3
4	Acua	491	1789 Saint-Denis Parkway	4
5	Adana	332	663 Baha Blanca Parkway	5
6	Addis Abeba	397	614 Pak Kret Street	6
7	Aden	214	751 Lima Loop	7

דוגמה #1

- אותה שאילתה באמצעות USING:

```
SELECT
        t1.city_id,t1.city,
        t2.address_id,t2.address,t2.city_id
FROM
city t1
INNER JOIN address t2
USING(city_id);
```

דוגמה #2


country
country_id SMALLINT
country VARCHAR(50)
last_update TIMESTAMP
Indexes
city
city_id SMALLINT
city VARCHAR(50)
country_id SMALLINT
last_update TIMESTAMP
Indexes
address
address_id SMALLINT
address VARCHAR(50)
address2 VARCHAR(50)
district VARCHAR(20)
city_id SMALLINT
postal_code VARCHAR(10)
phone VARCHAR(20)
location GEOMETRY
last_update TIMESTAMP
Indexes

דוגמה #2

חיבור של 3 טבלאות:

```
SELECT
t1.city_id,t1.city,t2.address_id,t2.address,t2.
city_id,
t1.country_id, t3.country_id,t3.country
FROM
city t1
INNER JOIN address t2
ON
t1.city_id = t2.city_id
INNER JOIN
country t3
ON
t1.country_id = t3.country_id;
```

INNER JOIN USING OTHER OPERATORS

- So far, you have seen that the join condition used the equal operator (=) for matching rows.

- In addition to the equal operator (=), you can use other operators such as greater than (>), less than (<), and not-equal (<>) operator to form the join condition.

```
SELECT
        t1.city_id,
        t1.city,
        t2.address_id,t2.address,
        t2.city_id,t1.country_id, t3.country_id,t3.country
FROM
city t1
INNER JOIN address t2
ON      t1.city_id = t2.city_id
INNER JOIN country t3
ON      t1.country_id = t3.country_id
AND   t3.country <> 'Argentina'
```

LEFT JOIN

- The left join clause selects data starting from the left table (t1).
- It matches each row from the left table (t1) with every row from the right table(t2) based on the join_condition.

- If the rows from both tables cause the join condition evaluates to TRUE, the left join combine columns of rows from both tables to a new row and includes this new row in the result rows.

- In case the row from the left table (t1) does not match with any row from the right table(t2), the left join still combines columns of rows from both tables into a new row and include the new row in the result rows.
- However, it uses **NULL** for all the columns of the row from the right table.

- In other words, left join returns all rows from the left table regardless of whether a row from the left table has a matching row from the right table or not.
- If there is no match, the columns of the row from the right table will contain
  NULL.

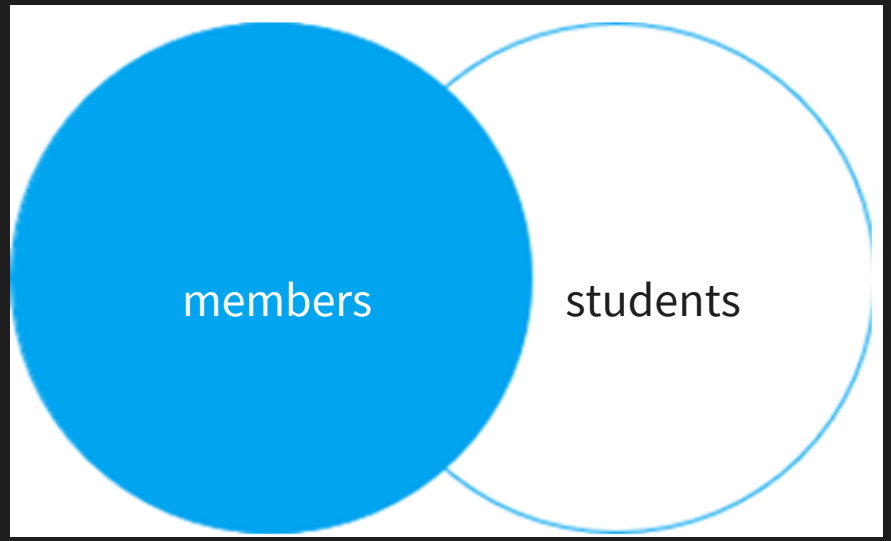
BASIC SYNTAX OF THE LEFT JOIN

- The following statement shows how to use the left join clause to join the two tables:

```
SELECT
  select_list
FROM
  t1
left join t2 ON
  join_condition;
```

דוגמה #1

```
SELECT
        m.member_id,
        m.name AS member,
        c.student_id,
        c.name AS 'student name'
FROM
members m
left join
students c using(name);
```



member_id	member	student_id	student name
1	John	1	John
3	Mary	2	Mary
5	Amelia	3	Amelia
2	Jane	NULL	NULL
4	David	NULL	NULL

דוגמה #2

- הדוגמה הבאה מראה כיצד להשתמש ב LEFT JOIN על מנת למצוא רשומות לא תאומות.
- השימוש ב LEFT JOIN מאוד יעיל כשרוצים לאתר רשומות הקיימות בטבלה אחת ולא בשנייה (לפי תנאי מסויים).

דוגמה #2

The following query uses the left join to find customers who didn't rent a film:

```
SELECT
C1.first_name,
C1.last_name,
R1.rental_date
FROM customer C1
LEFT JOIN rental R1
ON
C1.customer_id = R1.customer_id
WHERE R1.customer_id    is null;
```

	first_name	last_name	rental_date
▸	boaz	sch	NULL

RIGHT JOIN

- The right join clause is similar to the left join clause except that the treatment of tables is reversed.
- The right join starts selecting data from the right table instead of the left table.

- The right join clause selects all rows from the right table and matches rows in the left table.
- If a row from the right table does not have matching rows from the left table, the column of the left table will have NULL in the final result set.

BASIC SYNTAX OF THE RIGHT JOIN

- Here is the syntax of the right join of two tables t1 and t2:

```
SELECT select_list
  FROM t1
right join t2 ON
    join_condition;
```

- In this syntax:
  - t1 is the left table and t2 is the right table
  - join_condition specifies the rule for matching rows in both tables.

RIGHT JOIN AND THE USING CLAUSE

- If the join condition uses the equal operator (=) and the joined columns of both tables have the same name, you can use the using syntax:

```
SELECT
   select_las
t  FROM t1
right join t2 using(column_name);
```

RIGHT JOIN AND THE USING CLAUSE

- So, the following join conditions are equivalent:

  ON t1.column_name = t2.column_name

- and

  using (column_name);

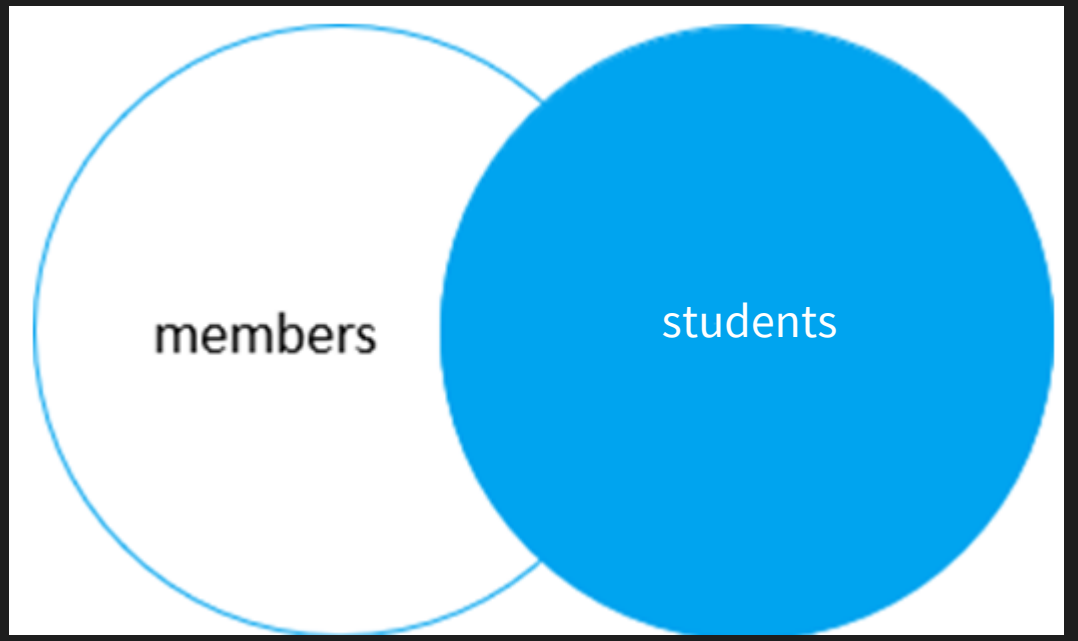
- To find rows in the right table that does not have corresponding rows in the left table, you also use a WHERE clause with the **IS NULL** operator:

```
SELECT
column_list FROM
table_1
right join table_2 using (column_name)
WHERE column_table_1 IS NULL;
```

דוגמה #1

This statement uses the right join to join the members and students tables:

```
SELECT
    m.member_id,
    m.name as member,
    c.student_id,  c.name
    student
FROM
    members m
right join students c
on c.name = m.name;
```

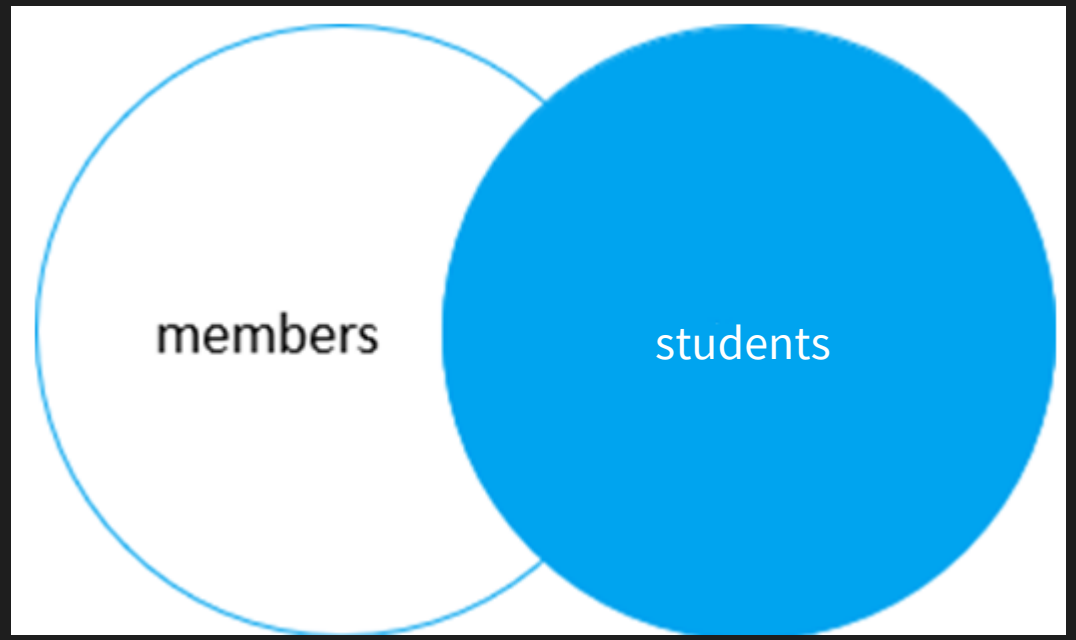


member_id	member	student_id	student
1	John	1	John
3	Mary	2	Mary
5	Amelia	3	Amelia
NULL	NULL	4	Joe

דוגמה #2

To find the students who are not in the members table, you use this query:

```
SELECT
    m.member_id,
    m.name as
    member,
    c.student_id,
    c.name
    student_name
FROM
    members m
right join students c using(name)
WHERE m.member_id IS NULL;
```



member_id	member	student_id	student_name
NULL	NULL	4	Joe

CROSS JOIN

- בניגוד ל JOINS שראינו עד עכשיו, ב JOIN CROSS אין שימוש בתנאי על מנת להשוות.
- ה CROSS JOIN מחבר כל רשומה מהטבל הראשונה עם רשומה מהטבלה השנייה.
- אם בטבלה אחת יש N רשומות ובטבלה השנייה יש M רשומות – שימוש ב JOIN CROSS יחזיר N*M רשומות

BASIC SYNTAX OF THE CROSS JOIN

- The following illustrates the syntax of the CROSS join clause that joins two tables t1 and t2:

```
SELECT * FROM t1
CROSS join t2;
```

- Note that different from the inner join, left join , and right join clauses, the CROSS join clause does not have a join predicate.
- In other words, it does not have the ON or using clause.

CROSS JOIN

- If you add a WHERE clause, in case table t1 and t2 has a relationship, the CROSS join works like the inner join clause as shown in the following query:

```
SELECT *
FROM t1
CROSS join t2
WHERE t1.id = t2.id;
```

דוגמה #1

This example uses the cross join clause to join the members with the memmbers tables:

```
SELECT
    m.member_id,
    m.name as  member,
    c.student_id,
    c.name student
FROM
    members m
CROSS join students c;
```

member_id	member	student_id	student
1	John	1	John
1	John	2	Mary
1	John	3	Amelia
1	John	4	Joe
2	Jane	1	John
2	Jane	2	Mary
2	Jane	3	Amelia
2	Jane	4	Joe
3	Mary	1	John

EXERCISES

- Return all **addresses** from the **city** ‘Lethbridge’.
- Return the full names (first and last) of all the **actors** and **costumers** that have the same first name
- Which **customers** live in the **address**: *1913 Hanoi Way,* city: *Sasebo*?
- Return the title and description of films from “Family” category (film, film_category, category)
- Add the actor list (first_name, last_name) for the above query (actor, film_actor) – order by film name, actor name